Current

Architecture blueprint — how it is built, and why

Every AI story that matters, from the labs and the industry press, summarised in plain language and updated hourly. This document explains the whole system: each component, the reasoning behind it, how a story travels from an RSS feed to a reader's screen, and what has already gone wrong in production.

Version	Date	Web	API
1.0	7 September 2026	current-3wkv.onrender.com	ai-pulse-api-zcyv.onrender.com

Contents

1 · What Current is

> An AI news reader that does the filtering for you. It watches 43 sources, discards duplicates and off-topic noise, and has a language model write a short plain-English summary of what survives.

The problem it solves is not access to AI news — there is far too much of it. The problem is that following the field properly means checking a dozen lab blogs and a dozen news sites, most of which repeat each other. Current collapses that into one timeline where each story appears once, described in a couple of sentences, with the genuinely significant ones marked.

What a reader can do

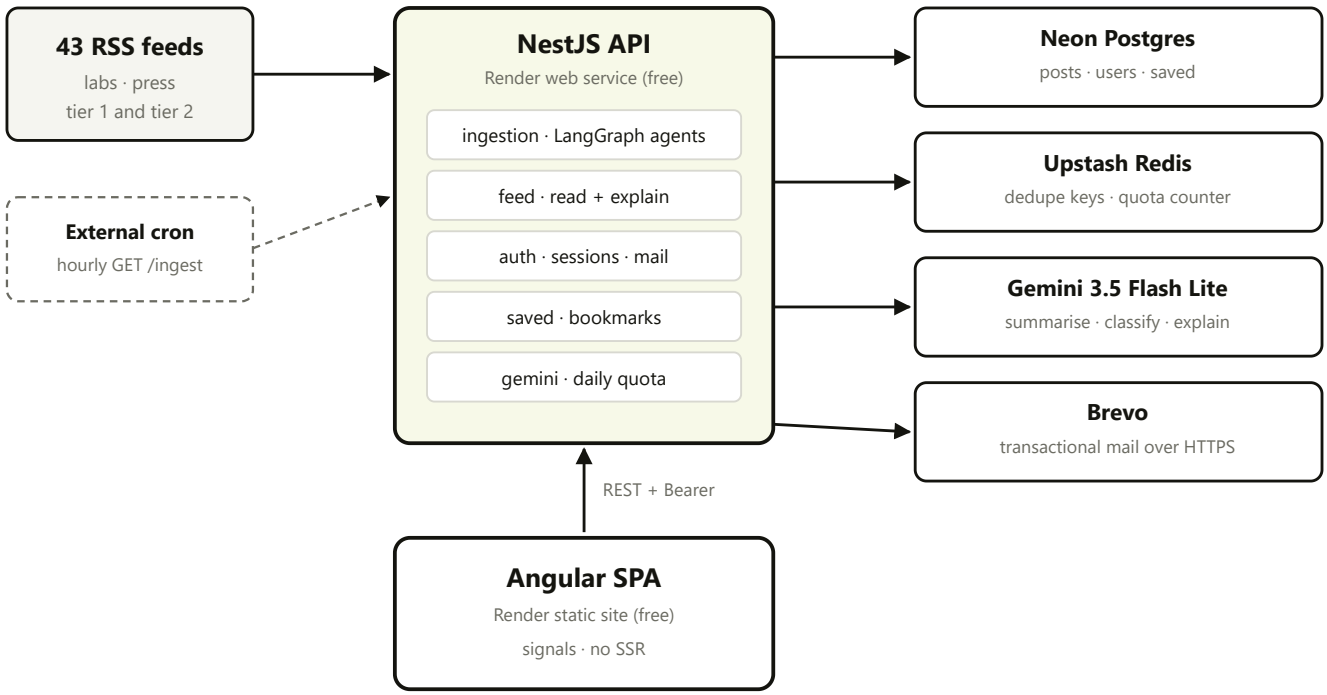
CAPABILITY	DETAIL
Read the feed	Newest first, infinite scroll, three layout modes — grid, continuous and one-at-a-time.
Filter	By source (multi-select), by search term, or jump to a specific date.
See what matters	Stories judged *major* at ingestion time are visually marked and drive the trending panel.
Go deeper	"Read more" generates a longer breakdown on demand, grounded in the original article text, then caches it forever.
Save	Signed-in readers bookmark posts; the account page lists them.

THE CONSTRAINT THAT SHAPED EVERYTHING

Current runs entirely on free tiers — hosting, database, Redis, AI and mail. That is not a detail, it is the central design constraint. It is why there is a daily AI budget with its own accounting, why ingestion is prioritised rather than exhaustive, why mail goes over an HTTP API rather than SMTP, and why the ingestion schedule lives outside the application. Nearly every unusual decision in this document traces back to it.

2 · System map

> Two deployed services, four external providers, one direction of data flow.



Solid: request path. Dashed: scheduled trigger, outside the app.

The API is the only component with credentials. The browser never talks to Postgres, Redis, Gemini or Brevo.

Why the frontend is a static bundle

The Angular app compiles to plain files served from a CDN — no Node process, no server-side rendering. It cannot go down independently of the CDN, costs nothing to host, has no cold start, and cannot leak a secret because it never holds one. The cost is that the first paint shows a loading state while it fetches the feed, and that search-engine crawlers see an empty shell. For a signed-in reading app, that trade is clearly worth it.

3 · The stack

> Every dependency here earns its place. What follows is the reason each one was chosen over the obvious alternative.

Frontend

CHOICE	WHY, AND WHAT WAS REJECTED
Angular 21	Standalone components with *signals* for state. Signals give fine-grained reactivity without pulling in a state-management library — the feed, filters and auth state are all plain signals, and the template re-renders only what changed.
No UI library	Badge, button, card, nav-item and page-loader are hand-built in `shared/ui`. A component library would have brought a design language that had to be fought; the whole visual identity here is bespoke.
Custom masonry	CSS cannot do true masonry yet. `grid-template-rows: masonry` is still behind a flag, and CSS multi-column fills top-to-bottom — which put the second-newest story at the bottom-left instead of beside the newest, destroying reading order. The directive measures each card and assigns it a span across many 4px row tracks.

Backend

CHOICE	WHY, AND WHAT WAS REJECTED
NestJS 12	Modules and dependency injection as a default rather than a convention. With ingestion, auth, feed and saved in one process, enforced boundaries are what stop them growing into each other.
Prisma 6 + Neon	Typed queries and tracked migrations — six so far, each reviewable. Neon's free tier is real Postgres. The schema uses a pooled connection at runtime and a direct one for migrations, which is Neon's documented requirement.
Upstash Redis	Two jobs only: 30-day URL dedupe keys and the daily AI counter. Chosen over node-redis because it speaks HTTP — no connection pool to keep alive on an instance that sleeps, and no socket to leak.
LangGraph	Each source runs a small state graph (`fetch → process → END`) rather than a loop body. The value is isolation: one feed timing out fails its own graph and the run continues. It also leaves room for per-source branching later without restructuring.
Gemini 3.5 Flash Lite	Summarising a headline and a blurb is not a hard reasoning task; it is a high-volume one. Flash Lite is the cheapest model that writes acceptable prose, and volume — not quality — is the binding constraint on the free tier.
Brevo over HTTP	Not a preference — a necessity. Render blocks outbound SMTP ports 25, 465 and 587 on free web services, so no SMTP client of any kind can send from there. Brevo's transactional API is HTTPS on 443, which no port policy touches. Its free tier verifies a single sender address, so no domain purchase is required.
Hand-rolled rate limiting	A plain `CanActivate` guard rather than `@nestjs/throttler`, matching the existing auth guard convention and avoiding a dependency for roughly forty lines of logic.

4 · Ingestion

> Turning 43 noisy feeds into a clean timeline, on a budget that can run out.

How the sources are described

Every source is a typed record, not just a URL. The extra fields exist because real feeds misbehave in specific, repeatable ways.

FIELD	WHAT IT SOLVES
tier	1 = OpenAI, Anthropic, DeepMind, Google. 2 = everyone else. Feeds into the *major* judgement.
priority	1–4. Processing order when the shared budget is tight, so an official announcement is never starved by an aggregator.
topicFilter	A keyword regex for multi-topic feeds, so a general tech feed doesn't dilute the timeline with unrelated posts.
maxPerRun	Aggregate search feeds return 100 items each. Without a cap, one publisher would crowd out every other source.
stripPublisherSuffix	Google News titles arrive as "Headline - Publisher". That suffix showed in the UI *and* defeated near-duplicate detection, because the same story from two publishers no longer looked alike.
deriveSourceFromTitle	Credits the real publisher — Reuters, The Verge — instead of labelling dozens of posts with one aggregator's name.

One run, step by step

1. **Trigger**
   An external scheduler calls `GET /ingest?key=…` hourly. The endpoint returns immediately rather than awaiting the run, because a full pass takes minutes and most schedulers time out long before that — reporting failures for runs that actually succeeded.

2. **Overlap guard**
   A flag rejects a second run while one is in progress. Without it, a slow run overlapping the next trigger would spend the same daily budget twice.

3. **Budget**
   One shared allowance of 20 summaries for the whole run, spent in priority order. Per-source caps alone, across 43 sources, would multiply well past the free tier.

4. **Fetch**
   Each source's graph pulls its feed with a polite bot User-Agent — several publishers reject requests without one.

5. **Filter**
   Items older than 7 days are dropped, the topic regex is applied, and at most 3 items per source per run survive.

6 **Dedupe, layer one**
The URL is checked against Redis, where every ingested URL is remembered for 30 days. Exact repeats stop here, costing nothing.

7 **Dedupe, layer two**
The headline is compared against the last 3 days of stored titles by significant-word overlap. Above 0.65 similarity it is treated as the same story — this is what stops one announcement appearing five times under five mastheads.

8 **Summarise and classify**
One Gemini call per surviving item produces both the plain-language summary and the *major* judgement together. Combining them halves the call count.

9 **Pace**
4.5 seconds between summaries, keeping the run under the model's per-minute limit. The daily limit is handled separately.

10 **Store**
The post is written with its source, tier, summary and major flag. The URL is unique in the schema, so a duplicate that slips through both layers is still rejected by the database.

WHY THE SCHEDULE LIVES OUTSIDE THE APPLICATION

There is an in-process hourly cron, and it is not the one that matters. A free Render instance sleeps after about 15 minutes of inactivity, and a sleeping process cannot run its own schedule. The external trigger is what actually keeps the feed fresh; the internal one covers local development and any host that stays warm. Both call the same code path, so they cannot drift apart.

5 · The AI budget

> Gemini's free tier caps requests per day as well as per minute. Nothing in a normal web architecture accounts for a dependency that simply stops answering two-thirds of the way through a Tuesday.

The accounting

LIMIT	VALUE	REASONING
Daily ceiling	800 calls	Deliberately under the real free-tier cap, so the app stops itself with headroom instead of discovering the limit by hitting it.
Ingestion ceiling	600 calls	Background work is cut off first.
Reserved	200 calls	Kept for user-triggered "Read more". **A background job must never be the reason a person tapping a button gets an error.**
Per run	20 summaries	24 hourly runs × 20 = 480 worst case, comfortably inside 600.
Pacing	4.5s between calls	Keeps a run under the per-minute limit.

Three details that matter more than they look

It increments, then compares. The counter is incremented first and the result checked afterwards, so two concurrent callers cannot both be told yes for the same final slot. Checking first and incrementing after is the classic version of this bug.

It fails open. If Redis is unreachable the call is allowed. Redis being down should not take the product down with it; the per-minute pacing and per-run caps still apply, so the worst case is a day that runs closer to the true ceiling.

The key is the UTC date. It matches how Google resets the quota, and carries a 48-hour expiry so it cleans up after itself.

> **THE ENDPOINT THAT SPENDS MONEY**
>
> `GET /feed/:id/detail` is public, and a post without a cached breakdown triggers a model call. Left unguarded, anyone could walk the post IDs and burn the entire daily budget in minutes — stopping ingestion and breaking "Read more" for every real reader until the UTC day rolled over. It is rate limited at 30 requests per 15 minutes per address: generous for someone genuinely reading, useless for draining a 600-call reserve. Results are cached in the database permanently, so each post costs at most one call ever.

6 · The reading path

> What actually happens between opening the app and seeing stories.

1. **The shell loads**
   A static bundle from the CDN. It immediately asks the API who the visitor is.

2. **Session check**
   `GET /auth/me` with the stored token. Until it answers, a loader shows — the app never renders a signed-out view to someone who turns out to be signed in.

3. **First page**
   `GET /feed?limit=20`. Ordered newest first.

4. **Scroll**
   An IntersectionObserver on a sentinel element near the bottom triggers the next page — no "load more" button.

5. **Next page**
   The cursor is the last row's `publishedAt` *and* its `id`.

6. **Layout**
   The masonry directive measures each card and assigns a row span, so cards pack upward within their column instead of every card in a row waiting for the tallest.

Why the cursor needs two halves

Paging on a timestamp alone assumes timestamps are unique. They are not: publishers post in batches, and several sources supply only a date, so ties are routine. With `publishedAt < before`, every tied row straddling a page boundary either disappeared or repeated — silently, with no error anywhere. The reader simply never saw those stories. The fix adds the ID as a tiebreak to both the cursor *and* the ordering, since a tiebreak the sort does not share is no tiebreak at all.

```
WHERE  publishedAt < :before
   OR (publishedAt = :before AND id < :beforeId)
ORDER BY publishedAt DESC, id DESC
```

Read endpoints

ENDPOINT	PURPOSE
GET /feed	Paged feed. Also serves search (`q`) and date-jump (`date`); source filter applies to all three.
GET /feed/sources	Distinct source names, for the filter checkboxes.
GET /feed/trending	This week's major-flagged posts, newest first, capped at 6.
GET /feed/has-updates	Whether anything arrived since a timestamp — powers the "new stories" nudge without refetching the feed.
GET /feed/:id/detail	The long breakdown. Cached after first generation; rate limited.

How "Read more" is grounded

The explainer does not work from the headline alone. It fetches the original article, strips the HTML to text, truncates to 8,000 characters and hands that to the model as grounding — with an 8-second timeout, after which it degrades to explaining from the title and summary rather than failing. A breakdown built from real article text is substantially better than one imagined from a headline, and the timeout means a slow publisher costs a worse answer, not an error.

7 · Accounts and mail

> Two ways in, one session format, and email that has to actually arrive.

Two sign-in paths

Google Sign-In verifies the Google ID token server-side and creates or finds the user. There is no client secret because this is the browser-side Google Identity flow — the client ID identifies the app, and the token's signature is what proves the claim. **Email and password** uses bcrypt, with passwords capped at 72 bytes because bcrypt silently truncates beyond that — without the cap, two different long passwords sharing a prefix would hash identically.

Sessions

Both paths issue the same JWT, valid 30 days, returned two ways: as an httpOnly cookie *and* in the response body. The duplication is deliberate. The API and the web app live on different `onrender.com` subdomains, and that domain is on the public suffix list — so the cookie is genuinely third-party there, which Safari and every iOS browser drop outright. The body token is stored and sent as a Bearer header, which works regardless. The cookie remains the preferred path where browsers still honour it, and it is what the email-verification redirect depends on, since a link click is a plain navigation with no JavaScript to attach a header.

Token lifetimes

TOKEN	LIFETIME	WHY
Session JWT	30 days	Long enough that a casual reader is not asked to sign in repeatedly.
Verification	24 hours	Enough to survive a spam folder and a night's sleep.
Password reset	1 hour	Deliberately shorter: a reset link is a live credential for taking over an account, so it should expire well before a signup link would.

Where verification links must point

The link in a verification email goes to the *API*, not the web app. This is not obvious and was originally wrong: a link built from the web origin loaded the single-page app, matched no route, and fell through to the home page — so the token was never consumed while the user watched a successful-looking redirect. The API consumes the token, then redirects to the web app.

Anti-enumeration

`forgot-password` and `resend-verification` return an identical response whether or not the account exists, whether or not it is already verified, and whether or not the mail send succeeded. Otherwise either endpoint becomes a tool for discovering who is registered. The consequence is that a genuine send failure is invisible to the user by design — so it is logged instead, and `/health/mail` exists to check the mail path directly.

8 · Data model

> Three tables. The schema is small on purpose.

POST	
id	cuid
source, sourceTier	Display name and 1/2 tier
title, url, summary	`url` is unique — the last line of defence against duplicates
isMajor	Set at ingestion by the classifier; drives highlighting and trending
publishedAt, createdAt	Publication time versus ingestion time — the feed sorts on the former
detailBreakdown	Nullable. The cached "Read more" text, written on first request

USER	
googleId	Nullable and unique. Postgres permits many NULLs in a unique index, so password-only users coexist fine
email	Unique, normalised before storage
passwordHash	Nullable — a Google-only account has no password, which is why password reset is a deliberate no-op for them
emailVerified	Gates email/password login
verificationToken, resetToken	Both unique and both expiring; consumed on use so links cannot be replayed

SAVEDPOST	
userId, postId	Unique together — saving twice is idempotent
cascade	On both sides: deleting a user drops their saved list, deleting a post drops the now-dangling bookmarks

Indexes exist on `publishedAt` (every feed query sorts by it) and `isMajor` (trending). Migrations are tracked in the repo — six to date, from the initial schema through user accounts, saved posts and password reset.

9 · Security posture

> What is actually defended, and by what.

CONCERN	DEFENCE
Password guessing	10 attempts per endpoint per address per 15 minutes. Keyed per endpoint, so a burst of signups cannot lock the same person out of logging in.
Account enumeration	Identical responses from the reset and resend endpoints regardless of outcome.
Password storage	bcrypt, minimum 8 characters, capped at 72 bytes.
Budget exhaustion	The one endpoint that spends money is rate limited and permanently caches its output.
Third-party cookies	Bearer token fallback, so auth does not depend on a browser policy.
XSS from model output	Model text is rendered through Angular interpolation, which escapes by default. There is no `innerHTML` anywhere in the app.
Unauthorised ingestion	A rotatable shared secret. It grants only "fetch the news early", which is why accepting it in a query string was an acceptable trade for a trigger that schedulers can actually call.
Secrets in the repo	`.env` is gitignored; a documented `.env.example` carries names only.

THE SUBTLE ONE: WHAT A RATE LIMIT IS KEYED ON

Rate limiting is only as good as its bucket key, and this is where it silently failed for the entire life of the deployment. Render fronts services with Cloudflare, so requests arrive having crossed three hops — client, Cloudflare edge, Render's internal router. Express, configured to trust one proxy, resolved the caller to that internal address, which varies between requests. Every request landed in its own fresh bucket. Fourteen consecutive password-reset attempts against production all succeeded; thirty-three consecutive detail requests all succeeded. The guard was present in code review and did nothing at runtime — the worst way for a security control to fail. It now keys on `cf-connecting-ip`, which Cloudflare overwrites and a client therefore cannot forge, and both limits were re-tested against production until they returned 429.

10 · Deployment and operations

Topology

SERVICE	TYPE	NOTES
ai-pulse-api	Render web service, free	Blueprint in `render.yaml`. Sleeps after ~15 minutes idle.
current-3wkv	Render static site, free	No rewrite rule needed — the build copies `index.html` to `404.html`, so deep links work as part of the build rather than as dashboard configuration.
Scheduler	External cron	Hourly `GET /ingest?key=…`

The three health endpoints

Each exists because a specific class of problem was, at some point, only diagnosable by reading logs.

ENDPOINT	ANSWERS
/health/config	Which environment variables are set (booleans only — never values), which commit is serving, and today's AI spend. The commit field distinguishes "the fix has not deployed yet" from "the fix did not work", which otherwise look identical.
/health/mail	Whether the mail key *works* — it asks the provider rather than checking that a variable is non-empty. Returns a truncated hash of the running key so two deployments can be compared without exposing it.
/health/ip	What the server resolves the caller to. Reflects the requester back at themselves, and turns "is rate limiting working" into one request instead of an inference.

Continuous integration

GitHub Actions builds both workspaces on every push and pull request. It deliberately builds rather than tests: there is no meaningful suite yet, and a green check that only ran `echo` would be worse than none. Both builds do catch the class of mistake that has actually reached production here — a type error, or a template calling a method that does not exist.

11 · Production failure log

> Five failures, all found in production, none by reading the code. They are recorded because the pattern they share is more useful than any of them individually.

1 · MAIL THAT NEVER LEFT THE BUILDING

Verification emails silently failed with `ENETUNREACH` on an IPv6 address. Three causes stacked: Render instances have no IPv6 route; the mail library resolves hostnames itself and picks from the combined A and AAAA results *at random*, so the standard Node fix could not reach it; and underneath both, Render blocks outbound SMTP ports entirely on free services, so no SMTP client could ever have worked. Each fix revealed the next.

Resolution: mail moved to an HTTP API on port 443.

2 · A VERIFICATION LINK THAT VERIFIED NOTHING

The link pointed at the web app, which has no matching route. Clicking it loaded the app, matched nothing, and landed on the home page — a successful-looking redirect that consumed no token, so the account stayed unverified and the user could not log in.

Resolution: build the link from the API's own origin, and report that origin in `/health/config` so a wrong one is visible without sending mail to find out.

3 · RATE LIMITING THAT NEVER LIMITED

Described in section 9. Present in the code, keyed on an address that changed every request, therefore inert — for the entire life of the deployment, on the endpoints that guard credentials.

Resolution: key on the Cloudflare-supplied client address, then re-test against production until it actually returned 429.

4 · PAGINATION THAT LOST STORIES

A cursor on `publishedAt` alone dropped every tied row that straddled a page boundary. No error, no log line — readers just never saw those posts.

Resolution: a composite `(publishedAt, id)` keyset cursor, verified by checking two live pages for overlap and gaps.

5 · NAVIGATION THAT LOOKED INTERACTIVE

The sidebar's Feed item rendered as a nav item and highlighted itself as active, but had no link and no click handler — so the account page had no way back. Separately, sign-out cleared the user only after the network round-trip, so the app rendered the feed for the length of that request before bouncing to the login page.

Resolution: a real router link, and clearing session state synchronously so the redirect happens on the same tick.

THE PATTERN

Four of these five looked correct in the source. They were controls and links that existed, compiled, and did nothing — and every one was found by exercising the deployed system rather than by reading it. The health endpoints and the production verification steps described throughout this document are the direct response: the goal is that "configured" and "working" can always be told apart by one request.

12 · Limits and what comes next

Known limits

LIMIT	WHEN IT WILL MATTER
Search is `ILIKE '%q%'`	Unindexable, so it is a full scan. Fine at hundreds of rows; wants Postgres full-text search at tens of thousands.
Single-instance assumptions	Rate limit buckets, the in-process cron and the overlap guard all live in memory. Correct on one instance; scaling out would double-spend the AI budget and halve the effective rate limit. Moving them to Redis is the fix.
Cold starts	A free instance sleeps after ~15 minutes, and the next visitor waits roughly 50 seconds. Mitigated by a keep-warm ping; removed entirely by a paid instance.
Test coverage	CI builds both apps, which catches type and template errors — but nothing yet catches "deployed and silently inert", which is precisely the failure mode this project keeps producing.
Mail deliverability	Mail is DKIM-signed by the relay rather than the sending domain, so a share of it is filed as spam. A verified custom domain would fix it properly.
No error tracking	Diagnosis means reading host logs by hand.

Where the next effort belongs

1. **Integration tests against a running instance.** Every failure in section 11 would have been caught by one. This is the single highest-value addition to the project.
2. **Error tracking.** Swallowed-and-logged failures are invisible until someone goes looking.
3. **Full-text search** before the post count makes the current approach visibly slow.
4. **Redis-backed rate limiting** whenever a second instance becomes plausible.

Current — architecture blueprint, version 1.0, 7 September 2026. Generated from the repository at commit `7a0dcec`. Every figure in this document was read from the code rather than recalled, and every production claim was verified against the live deployment.